# Rust Verification Through Lean 4 Tooling Investigation report

## Discovery report for Milestone 3 — Implementation and Verification Ethereum

Delivered: April 02, 2026

runtime verification

# Table of Contents


- Introduction
- Our Contributions
  - fold_arity: FRI Round-Scheduling Verification
    - BasicProofs.lean — fully proved (0 sorry)
    - FoldingCorrectness.lean — ArkLib bridge
  - fold_step: Arity-2 FRI Folding Verification with ArkLib
    - BasicProofs.lean — fully proved (0 sorry)
    - FoldingCorrectness.lean — ArkLib connection
  - Mersenne31 Field Arithmetic Extraction and verification via Aeneas
    - Extraction status
    - Aeneas-specific issues encountered
    - Mersenne31 `add` and `mul` verification — CompPoly Bridge
  - Upstream Contributions to Aeneas:
    - PR #832 — Rename BitVec.toNat_pow BitVec.ofNat_pow to avoid conflict with Batteries
    - PR #833 — Fix `overflowing_add` upper bound for `IScalar` s
- Conclusion and Future Work

# Introduction

This report covers Month 3 of the Rust-to-Lean formal verification project. During the first three months, we established the Hax-based pipeline and verified several cryptographic primitives from Plonky3 and RISC Zero (FRI folding, Horner evaluation, Merkle inclusion, 32-bit ADC). That work is collected in the rust-lean repository.

In Month 3 the focus shifted to Aeneas — an alternative extraction tool that operates on Rust MIR (via Charon) and produces cleaner Lean 4 code compared to Hax. The main achievements are:

- **fold_arity:** full verification of `compute_log_arity_for_round` (FRI round-scheduling logic) — 6 theorems, 0 sorry.
- **fold_step:** verification of the core **arity-2 FRI folding operation** with an **ArkLib integration** connecting the Rust implementation to formal polynomial specifications (`foldNth`, `splitNth`, `Domain`) — the first function in the pipeline where ArkLib is genuinely load-bearing.
- **Mersenne31 field arithmetic:** Extracted a faithful initial model of Plonky3's Mersenne31 prime field via Aeneas into Lean 4. Also proved bidirectional correspondence for the `add` and `mul` functions with CompPoly's `Mersenne31.Field` specification: `add_verify_to_spec` / `add_verify_of_spec` and `mul_verify_to_spec` / `mul_verify_of_spec` establish that the extracted operations agree with the formal field spec in `ZMod (2^31 - 1)`, with boundary cases (outputs equal to `2^31 - 1`) handled explicitly.
- **Upstream contribution to Aeneas:**
  - resolved the `BitVec.toNat_pow` naming collision between Aeneas and Batteries/mathlib (PR #832, merged).
  - Correct `overflowing_add` overflow upper bound for `IScalars` (PR #833, merged).

The end-to-end pipeline is:

```
Rust source  ──Charon──▸  LLBC  ──Aeneas──▸  Lean 4  ──proofs──▸  ArkLib specs
```

# Our Contributions

## fold_arity: FRI Round-Scheduling Verification

**Rust function:** `compute_log_arity_for_round` from Plonky3 fri/src/config.rs. Determines the folding arity for each FRI round, respecting the maximum arity constraint, distance to final height, and distance to the next input commitment.

**Pipeline:** Rust → Charon → Aeneas → Lean 4 ( `FoldArity.lean` ).

### BasicProofs.lean — fully proved (0 sorry)

| Theorem | Property |
|---|---|
| `bounded_by_max_log_arity` | result ≤ max_log_arity |
| `bounded_by_target_distance` | result ≤ current − final (None case) |
| `respects_next_input` | result ≤ current − next (Some case) |
| `takes_minimum_of_constraints` | result ≤ min(current−next, max_arity) |
| `none_equals_some_at_target` | None ≡ Some(final_height) boundary case |
| `monotonic_in_max_arity` | monotone in the max_log_arity cap |
| 3 concrete examples | specific input/output checks |

All proofs use standard Lean tactics + Aeneas's `progress` and `omega`, with **zero custom lemmas** — a notable improvement over the Hax pipeline which required workarounds documented in HAX_MISSING_LEMMAS.md.

### FoldingCorrectness.lean — ArkLib bridge

| Theorem | Status | Property |
|---|---|---|
| `compute_log_arity_gives_folding_degree` | Proved | result maps to folding degree |
| `folding_respects_final_height` | Stated | domain height ≥ final after fold |
| `folding_reduces_degree` | Stated | `foldNth` reduces degree by arity |
| `round_consistency_preserved` | Stated | consecutive rounds ≤ total distance |
| `arity_determines_domain_step` | Stated | subdomain containment via `evalDomain` |
| 2 concrete examples | Proved | `log_to_arity 7 = 128` , height arithmetic |

Since `compute_log_arity_for_round` is pure integer scheduling logic, the ArkLib connection is preparatory — ArkLib's polynomial specifications do not directly constrain a function that only computes natural numbers. This motivated the next contribution.

# fold_step: Arity-2 FRI Folding Verification with ArkLib

**Rust function:** `fold_step` — a standalone implementation of the mathematical core of Plonky3's `fold_matrix` (arity-2 case from `two_adic_pcs.rs`). Given evaluations `lo = f(x)` and `hi = f(−x)`, a challenge β, and precomputed inverses, it computes:

```
fold_step(lo, hi, β, x⁻¹, 2⁻¹, p) = ((lo + hi) + β · (lo − hi) · x⁻¹) · 2⁻¹  mod p
```

This equals `(foldNth 2 f β).eval(x²)` — one evaluation of the folded polynomial. This is the operation performed in **every round** of the FRI protocol.

**Pipeline:** Rust → Charon → Aeneas → Lean 4 ( `FoldStep/Funs.lean` ).

## BasicProofs.lean — fully proved (0 sorry)

| Theorem | Property |
|---|---|
| `fold_step_nat_bounded` | Pure Nat spec always returns < p |
| `fold_step_spec` | **Key theorem:** Aeneas-extracted function matches `fold_step_nat` and result < p |
| `fold_step_succeeds` | Function never panics under `SafeFieldParams` |
| 4 concrete examples | `fold_step_nat` and Rust function agree on p = 7 and p = 17 |

The `fold_step_spec` proof steps through all 13 checked arithmetic operations in the Aeneas extraction using the `progress` tactic, proving that each intermediate addition/multiplication cannot overflow `u64` and the final result matches the pure specification.

## FoldingCorrectness.lean — ArkLib connection

| Theorem | Status | Property |
|---|---|---|
| `fold_step_field_eq` | **Proved** | Algebraic identity in `ZMod p` (by `ring`) |
| `fold_step_domain_shrinks` | **Proved** | Squaring map sends D_n → D_{n−1} |
| `fold_step_is_foldNth_eval` | Stated | `fold_step = (foldNth 2 f β).eval(x²)` |
| `fold_step_degree_halves` | Stated | `deg(foldNth 2 f β) ≤ deg(f)/2` |

The two stated theorems require evaluation lemmas for `splitNth` / `foldNth` from ArkLib that are not yet available; their proof sketches are documented inline. Closing them would be a contribution to ArkLib itself.

**Why ArkLib matters here:** Unlike `fold_arity` (pure integers), `fold_step` performs actual field arithmetic that directly corresponds to ArkLib's `foldNth 2 f β`. The verification establishes that the Rust implementation computes exactly one evaluation of the mathematically-defined FRI fold — the first result in this project where ArkLib specifications are genuinely load-bearing.

# Mersenne31 Field Arithmetic Extraction and verification via Aeneas

**Target:** Plonky3's `mersenne31.rs` and `field.rs`, with verification specifications aimed at CompPoly.

**Work location:** `Plonky3-Field-Arithmetic/Aeneas` in the `rust-lean` repo.

This work continues the field arithmetic strand from Phase 2 (which used Hax), now applying Aeneas as the extraction backend. Incremental Rust models are built in `src/`, mirroring the structure of the Hax models while adapting for Aeneas's MIR-based extraction.

**Pipeline:** Rust → Charon → Aeneas → Lean 4 ( `AeneasFieldArithmetic` ).

## Extraction status

The extraction pipeline is fully operational. `extract-aeneas.sh` runs Charon followed by Aeneas with `-backend lean -split-files`, then applies a set of documented post-

processing fixes to the generated Lean. The extracted output lives in `lean/AeneasFieldArithmetic/Generated/`.

## Aeneas-specific issues encountered

For a list of adaptations that were needed for the better extraction of our Rust model see here.

## Mersenne31 `add` and `mul` verification — CompPoly Bridge

Once we had a satisfactory-enough model that correctly extracts to Lean 4 we set out to test the proof ergonomics of the extracted Rust. To this end, we verified the extracted `add` and `mul` for Mersenne31. The proofs can be accessed here.

The proof work establishes a bidirectional correctness bridge between the Aeneas-extracted Mersenne31 Rust implementation and the `Mersenne31.Field` type from CompPoly.

`Interface.lean` defines the translation layer and core lemmas:

- `to_m31_spec` / `of_m31_spec` — mappings between the extracted `m31` struct (a wrapper around `U32`) and `Mersenne31.Field` (the CompPoly spec type, `ZMod (2^31-1)`).
- `add_logic` / `mul_logic_nat` — pure functional specs distilling the logic of the extracted operations, decoupled from the `Result` monad.
- `add_spec`, `mul_spec_nat` — prove the extracted functions equal their pure specs.

`AddVerify.lean` and `MulVerify.lean` prove the main correctness theorems:

| Theorem | Direction | Property |
|---|---|---|
| `add_verify_to_spec` | impl → spec | `to_m31_spec(add_logic(n, m)) = to_m31_spec(n) + to_m31_spec(m)` (non-degenerate: sum ≠ $2^{31}-1$) |
| `add_verify_of_spec` | spec → impl | `of_m31_spec(p + q) = ⟨add_logic(of_m31_spec(p), of_m31_spec(q))⟩` |
| `mul_verify_to_spec` | impl → spec | `to_m31_spec(mul_logic_nat(n, m)) = to_m31_spec(n) * to_m31_spec(m)` |
| `mul_verify_of_spec` | spec → impl | `of_m31_spec(p * q) = mul_logic_nat(of_m31_spec(p), of_m31_spec(q))` |

Proofs use `grind`, `omega`, `simp`, and `aesop`; no custom axioms. `AeneasUtils.lean` provides auxiliary `UScalar` / `IScalar` bitvector lemmas bridging Aeneas's scalar types with Lean's `Nat` arithmetic.

Given the presence of non-canonical zeros in the implementation field, some equivalence theorems had to exclude these cases when the result of the operation is a non-canonical zero on the implementation part.

## Upstream Contributions to Aeneas:

### PR #832 — Rename BitVec.toNat_pow BitVec.ofNat_pow to avoid conflict with Batteries

Aeneas and Batteries/mathlib both defined `BitVec.toNat_pow` and `BitVec.ofNat_pow`, causing a name collision that prevented importing both libraries in a single Lean file. We submitted PR #832 which removed the duplicate definitions from Aeneas and replaced them with imports from Batteries. The PR was merged on March 17, 2026.

This directly unblocked the `FoldingCorrectness.lean` files in both `fold_arity` and `fold_step`, which import ArkLib (and transitively Batteries).

### PR #833 — Fix `overflowing_add` upper bound for `IScalar`s

PR #833, merged March 15, 2026.

`IScalar.overflowing_add` used `2^ty.numBits` as the signed overflow upper bound instead of the correct `2^(ty.numBits-1)`, causing e.g. `overflowing_add .I8 127 1` to incorrectly report no overflow. Fixed `OverflowingOps.lean` to use the correct bound — a soundness fix relevant to any proof reasoning about checked signed arithmetic in the extracted code.

# Conclusion and Future Work

In this reporting period we extended the Rust-to-Lean verification pipeline from pure scheduling logic to actual cryptographic field arithmetic, demonstrating that Aeneas produces Lean 4 code suitable for integration with ArkLib's formal FRI specifications. The `fold_step` verification is, to our knowledge, the first formally verified connection between an Aeneas-extracted Rust FRI implementation and ArkLib's polynomial folding definitions.

Key results:

- **9 fully proved theorems** across two Lean projects (6 for fold_arity, 3 for fold_step) plus 7 concrete examples.
- **2 proved ArkLib-side theorems** ( `fold_step_field_eq` , `fold_step_domain_shrinks` ) and 2 precisely stated open goals.
- **1 merged upstream PR** resolving the Aeneas/Batteries name collision.
- **Successfully establish an extraction pipeline** for a faithful initial model of Plonky3's Mersenne31 prime field.
- **Real-world code verification** by verifying the verbatim implementation of Mersenne31's addition and multiplication functions.

**Future work:**

1. **Close the 2 remaining sorry goals** in `fold_step` by contributing `foldNth` evaluation lemmas to ArkLib (relating `foldNth 2 f β` evaluated at `x²` to `f(x)` and `f(-x)` ).
2. **Verify `fold_matrix`** — applying `fold_step` across all rows of an evaluation matrix — to cover the full Plonky3 FRI round.
3. **Verify `verify_query`** — the FRI verifier's per-query check, combining fold_step with commitment verification.
4. Continue closing sorry stubs in `fold_arity/FoldingCorrectness.lean` .
5. Add more features to the realistic Mersenne31 model for correct extraction.
6. Verify more real-world functions present in this model.